```
# ==========================================================
# Question 8: Sensor Noise and Noise Removal
# Google Colab Compatible
# ==========================================================

# Install OpenCV
!pip install opencv-python -q

import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# ----------------------------------------------------------
# Upload Image
# ----------------------------------------------------------
print("Upload an image (JPG/PNG)")

uploaded = files.upload()

filename = list(uploaded.keys())[0]

# Read image
img = cv2.imread(filename)

if img is None:
    print("Error: Image not loaded!")
else:

    # Convert BGR to RGB
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # ======================================================
    # Add Gaussian Noise (Sensor Noise)
    # ======================================================

    row, col, ch = img.shape

    mean = 0
    sigma = 25

    gaussian = np.random.normal(mean, sigma, (row, col, ch))
    noisy_gaussian = img + gaussian
    noisy_gaussian = np.clip(noisy_gaussian, 0, 255).astype(np.uint8)

    # ======================================================
    # Add Salt-and-Pepper Noise
    # ======================================================

    noisy_sp = img.copy()

    prob = 0.02

    num_salt = int(prob * img.size * 0.5)
    coords = [np.random.randint(0, i - 1, num_salt) for i in img.shape]
    noisy_sp[tuple(coords)] = 255

    num_pepper = int(prob * img.size * 0.5)
    coords = [np.random.randint(0, i - 1, num_pepper) for i in img.shape]
```

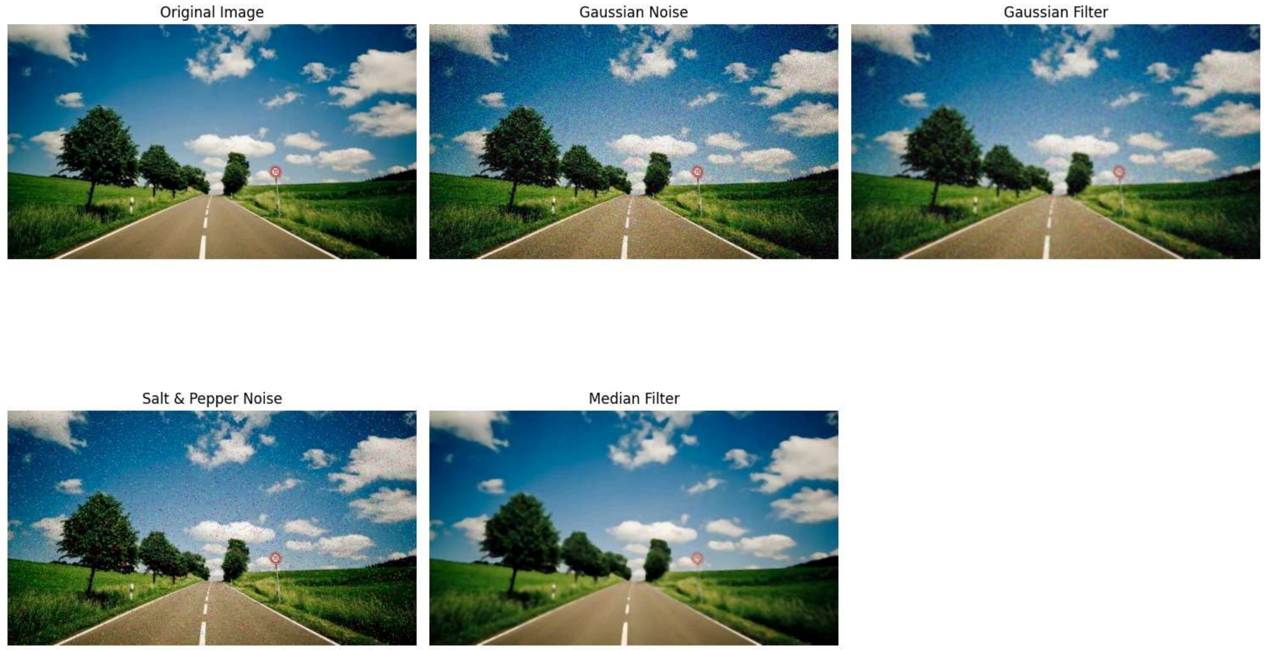
```
noisy_sp[tuple(coords)] = 0

# ======================================================
# Noise Removal
# ======================================================

# Gaussian Filter
gaussian_filtered = cv2.GaussianBlur(noisy_gaussian, (5,5), 0)

# Median Filter
median_filtered = cv2.medianBlur(noisy_sp, 5)

# ======================================================
# Display Results
# ======================================================

plt.figure(figsize=(15,10))

plt.subplot(2,3,1)
plt.imshow(img)
plt.title("Original Image")
plt.axis("off")

plt.subplot(2,3,2)
plt.imshow(noisy_gaussian)
plt.title("Gaussian Noise")
plt.axis("off")

plt.subplot(2,3,3)
plt.imshow(gaussian_filtered)
plt.title("Gaussian Filter")
plt.axis("off")

plt.subplot(2,3,4)
plt.imshow(noisy_sp)
plt.title("Salt & Pepper Noise")
plt.axis("off")

plt.subplot(2,3,5)
plt.imshow(median_filtered)
plt.title("Median Filter")
plt.axis("off")

plt.tight_layout()
plt.show()

# ======================================================
# Observation
# ======================================================

print("\nObservation:")
print("1. Gaussian noise simulates sensor electronic noise.")
print("2. Salt-and-pepper noise simulates faulty pixels.")
print("3. Gaussian Filter effectively reduces Gaussian noise.")
print("4. Median Filter effectively removes Salt-and-Pepper noise while preserving edges.")
print("\nConclusion:")
print("Noise introduced during image acquisition can be minimized using filtering techniques s
```

```
Upload an image (JPG/PNG)
```

Choose Files images.jpg

images.jpg(image/jpeg) - 33207 bytes, last modified: 7/14/2026 - 100% done

```
Saving images.jpg to images.jpg
```




```
Observation:
1. Gaussian noise simulates sensor electronic noise.
2. Salt-and-pepper noise simulates faulty pixels.
3. Gaussian Filter effectively reduces Gaussian noise.
4. Median Filter effectively removes Salt-and-Pepper noise while preserving edges.

Conclusion:
Noise introduced during image acquisition can be minimized using filtering techniques such as Gaus
```